

Managing Variables and Side Effects in Programming

Local vs. Global Scope

- Understanding the distinction between local and global scope is crucial in preventing side effects due to variable name conflicts.
- Modifications made to local variables within functions do not affect the global variables unless explicitly stated using the global keyword or if the function's return value is used to update the global variable.

Code with Side Effects

- Side effects occur when an operation modifies state outside its local environment.
- Debugging difficult because the behaviour of the program depends on the order and context of operations

```
# Global variable
count = 0

def increment():
    global count # Declare that we want to modify the global variable
    count += 1 # This modifies the global state

def main():
    print(f"Initial count: {count}") # Should be 0
    increment() # This will change the global 'count'
    print(f"Count after increment: {count}") # Should be 1

if __name__ == "__main__":
    main()
```

```
In [1]: runfile('C:/Users/Mireilla.DCS/.spy
Initial count: 0
Count after increment: 1

In [2]:
```

Global Variable: The variable **count** is defined outside of any function, making it a global variable. This means it can be accessed and modified from anywhere in the program.

Function with Side Effect: The function **increment()** is designed to modify the global variable **count**. It uses the *global* keyword to indicate that it intends to modify the global variable **count**, rather than creating a local variable of the same name. It alters this variable outside its local scope.

Modifying Global State: When **increment()** is called, it increments the value of **count**, which modifies the state outside its local environment (i.e., the global state).

Output: The **main() function** demonstrates the side effect by printing the value of **count** before and after calling **increment()**. You can see how calling **increment()** changes the global variable's value.

Code without Side Effects

To avoid side effects, you can return a new value rather than modifying global state.

In this code, **increment ()** returns a new value instead of altering a global variable, thus avoiding side effects and making the code easier to understand and maintain.

This practice is common in functional programming, which emphasises avoiding side effects to create more predictable and testable code.

```
@author: mireilla
"""
def increment(number):
    # Increment the given value and return the new value
    return number + 1 # No modification of global state; returns a new value

def main():
    # Use a local variable to hold the count
    local_count = 0

    # Print the initial value of the local count
    print(f"Initial local count: {local_count}") # Should be 0 initially

    # Update the local count by calling increment and assign the result back
    local_count = increment(local_count) # No global state change

    # Print the value of the local count after increment
    print(f"Local Count after increment: {local_count}") # Should be 1 after increment

if __name__ == "__main__":
    # Execute the main function when the script is run
    main()
```

```
In [7]: runfile('C:/Users/Mireilla.DC
Mireilla.DCS/OneDrive - University of
Initial local count: 0
Local Count after increment: 1
```

Another example of Code Side Effects

```
@author: mireilla
"""
def calculate_area(length, width):
    # Local variable 'area' inside the function scope
    # This local area is independent of the global area,
    #demonstrating how variables with the same name can exist
    #in different scopes without directly affecting each other.
    area = length * width
    print(f"Area inside function: {area}")
    return area

# Global variable 'area' is initialized to 0 outside of any function.
# This global variable is accessible throughout the program
area = 0

def main():
    # Local variable 'area' within main function
    area = calculate_area(5, 3) # This should be 15
    print(f"Area in main after calculation: {area}")

    # Conflicting variable name 'area' in another context.
    # the same variable name area is reused to store a different value (7).
    #This change affects only the area within the main function,
    #not the global variable or the variable within calculate_area.
    area = 7 # Assign a new value to 'area', this affects the local scope of main
    print(f"New value assigned to area in main: {area}")

if __name__ == "__main__":
    # Execute the main function
    main()

# Global 'area' variable is not affected by changes in the main function
# because all changes to area were local to the functions.
print(f"Global area after main execution: {area}")
```

```
In [9]: runfile('C:/Users/Mireilla.DCS/OneDrive -
OneDrive - University of Glasgow/ProgSD2024-2025'
Area inside function: 15
Area in main after calculation: 15
New value assigned to area in main: 7
Global area after main execution: 0
```

Challenges of Using Multiple Variables with the Same Name

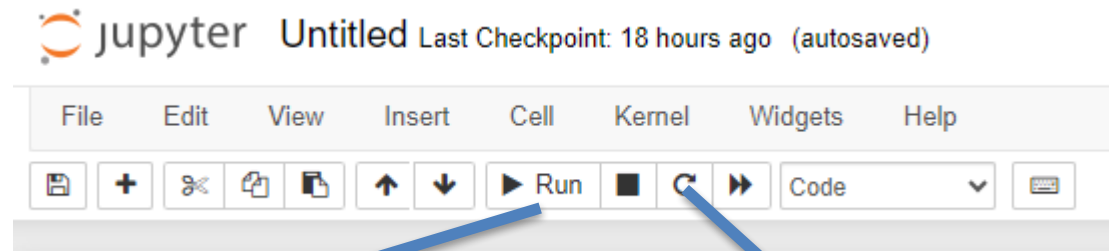
- **Leads to confusion** and difficult-to-trace bugs. Variables with the same name can exist in different scopes (e.g., global, function-level) without directly interfering with each other. However, this can lead to confusion if not carefully managed
- Use **descriptive and unique variable names**, especially in large programs or shared environments, to reduce the risk of name conflicts and unintended side effects.
- **Debugging Challenges:** Keep functions small and focused with clear input and output parameters to help maintain clarity and prevent side effects.

Challenges in Jupyter Notebooks

- Notebooks allow non-linear code execution, leading to inconsistent variable states.
- Variables can be overwritten across cells, causing unexpected results.
- Understanding the order of cell execution is critical.

Best Practices for Managing Variables

- Use unique and descriptive variable names to avoid conflicts.



When using Jupyter Notebook, Run cells sequentially and avoid jumping around without resetting the kernel.

Regularly reset the kernel to ensure code is executed in a clean state.

Conclusion and Key Takeaways

- Proper variable management is essential for robust code.
- Understanding side effects and state management helps avoid common pitfalls.
- Apply best practices consistently to ensure code reliability.